RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23331 - DESIGN AND ANALYSIS OF ALGORITHMS

Laboratory Record Note Book

Name : SURENDAR S

Year / Branch / Section : SECOND / IT / D

University Register No. : 2116251001276

Semester: III / THIRD

Academic Year : 2026 - 2027

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

NAME SURENDAR S

ACADEMIC YEAR 2026 - 2027 SEMESTER III BRANCH INFORMATION TECHNOLOGY

UNIVERSITY REGISTER NO. 2116251001276

Certified that this is the bonafide record of work done by the above student in the CS23331 Design and Analysis of Algorithms *Laboratory during the year* 2026 - 2027

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on .....

Internal Examiner

External Examiner

Exp No	Date	List of Experiments	QR Code	Teacher's Sign
1		**Finding the Algorithm's Time Complexity Using Counter Method** a. Problem 1 b. Problem 2 c. Problem 3 d. Problem 4 e. Problem 5		
2		**Divide and Conquer** a. Number of Zeroes in a Given Array b. Majority Element c. Finding Floor Value d. Two Elements Sum to x e. Implementation of Quick Sort		
3		**Greedy Algorithm** a. Coin Problem b. Cookies Problem c. Burger Problem d. Array Sum Max Problem e. Product of Array Elements - Minimum		
4		**Dynamic Programming** a. Playing with Numbers b. Playing with Chessboard c. Longest Common Subsequence d. Longest Non-Decreasing Subsequence		

5		**Competitive Programming** a. Finding Duplicates - O(n^2) Time & O(1) Space Complexity b. Finding Duplicates - O(n) Time & O(1) Space Complexity c. Print Intersection of 2 Sorted Arrays - O(m*n) Time & O(1) Space Complexity d. Print Intersection of 2 Sorted Arrays - O(m+n) Time & O(1) Space Complexity e. Pair with Difference - O(n^2) Time & O(1) Space Complexity f. Pair with Difference - O(n) Time & O(1) Space Complexity		

NAME: SUBENDAR. S

STD: II Yr / III Sem DIV: ROLL NO.: 25100276

SUBJECT: DAA OBSERVATION

INDEX

SR. NO.	DATE	TITLE	PAGE NO.	TEACHER'S SIGN
I		Finding the Time Complexity of Algorithms using counter method.		
1.	14/08/26	Problem 1 - finding Complexity		
2.	14/8/26	Problem 2 - finding Complexity		
3.	14/8/26	Problem 3 - finding complexity		
4.	14/8/26	Problem 4 - finding complexity		
5.	14/8/26	Problem 5 - finding complexity		
II		Divide and Conquer Method		
6.	21/08/26	Find the number of zeroes in an array		
7.	21/8/26	Majority Element.		
8.	21/8/26	Finding floor Value.		
9.	21/8/26	Two elements sum to x		
10.	21/8/26	Sorting - Quick Sort.		
III	28/08/26	GREEDY ALGORITHMS		
	28/8/26	1-G - Coin Problem		
	28/8/26	2-G - Cookies Problem		
	28/8/26	3-G - Burger Problem		
	28/8/26	4-G - Array Sum Max Problem		
	28/8/26	5-G - Product of array element - Minimum		
		DYNAMIC PROGRAMMING		
1	4/09/26	1-DP - Playing with Numbers		
2	4/9/26	2-DP - Playing with Chessboard.		
3	4/9/26	3-DP - Longest Common Subsequence.		
4	4/9/26	4-DP - Longest non-decreasing subsequence.		

SR. NO.	DATE	TITLE	PAGE NO.	TEACHER'S SIGN
		Competitive Programming.		
1	25/09/26	finding duplicates $(O(n^2), O(1))$		
2	25/9/26	finding duplicates $O(n), O(1)$		
3	25/9/26	Intersection of two array $O(m+n)$ $O(1)$		
4	25/9/26	Intersection of two array $O(m+n), O(1)$		
5	25/9/26	Pair with difference $O(n^2), O(1)$		
6	25/9/26	Pair with difference $O(n), O(1)$		

Completed

Finding the complexity using Counter Method.

Problem 1:

AIM: To convert the following algorithm into a program and find its time complexity using counter method.

Void function (int n)

Algorithm:

```
Void function (int n){
    int i=1;
    int s=1;
    while (s<=n){
        i++;
        s+=i;
    }
}
```

Program:

```
#include <stdio.h>
int main(){
    int n;
    int count =0;
    int i=1;
    count++;
    int s=1;
    count++;
    scanf("%d", &n);
    while (s<=n){
        count++;
        if (s>n){
            break;
        }
        i++;
        count++;
        s+=i;
        count++;
    }
    count++;
```

```
    printf("%d", count);
    return 0;
}
```

Sample Test Case

Input	Expected	Got
9	12	12
4	9	9

RESULT:

The program was written and executed successfully.

problem 2:

AIM: Convert the following algorithm into a program and find its time complexity using the counter method.

void function (int n)

Algorithm:

```
void func(int n){
    if (n==1){
        printf("*");
    }
    else{
        for (int i=1; i<=n ; i++){
            for (int j=1; j<=n; j++){
                printf("**");
                printf("**");
                break;
            }
        }
    }
}
```

program:

```
#include<stdio.h>
void func(int n){
    int count=0;
    count++;
    if (n==1){
        count++;
    }
    else{
        count++;
        for (int i=1; i<=n; i++){
            count++;
            for (int j=1; j<=n; j++){
                count++;
```

```
            count++;
            count++;
            break;
        }
        count++;
    }
}
printf("%d", count);
}
int main() {
    int n;
    scanf("%d", &n);
    func(n);
    return 0;
}
```

Sample Test Case

Input	Expected Output	Output
2	12	12
1000	5002	5002
143	717	717

RESULT: The program is written and executed successfully using counter method.

Problem 8:

AIM: Convert the following algorithm into a program and find its time complexity using counter method.

Algorithm:

```
Factor (num) {
    for (int i=1; i<= num; i++) {
        if (num%i==0) {
            printf("%d", i);
        }
    }
}
```

Program:

```
#include <stdio.h>
int factor (int num) {
    int count=0;
    for (int i=1; i<=num; ++i) {
        count++;
        if (num%i==0) {
            count++;
        }
        count++
    }
    printf ("%d", count);
    return 0;
}
int main() {
    int n;
    scanf("%d", &n);
    factor (n);
    return 0;
}
```

Sample Test Case:

Input	Expected Output	Output
12	31	31
25	54	54
4	12	12

RESULT: Thus the program was written and executed successfully using counter method.

Problem: 4

Aim: Convert the following algorithm into a program and find its time complexity using counter method.

Algorithm:

```
void function (int n){
    int c=0;
    for(int i=n/2; i<n; i++)
        for (int j=1; j<n; j=2*j)
            for (int k=1; k<n; k=k*2)
                c++;
}
```

Program:

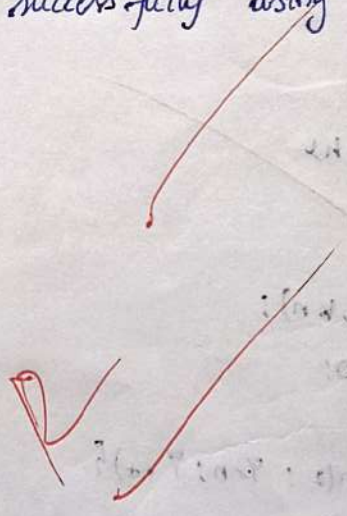
```
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int count=0;
    count++;
    for (int i=n/2; i<n; i++){
        count++;
        for (int j=1; j<n; j=j*2){
            count++;
            for (int k=1; k<n; k=k*2){
                count+=2;
            }
            count++;
        }
        count++;
    }
    count++;
```

```
    printf("%d", count);
    return 0;
}
```

Sample Test Case:

Input	Expected Output	Output
4	30	30
10	212	212

RESULT: Thus the program was written and executed successfully using counter method.



Problem 5:

AIM: Convert the following algorithm into a program and find its time complexity using counted method.

Algorithm:

```
void reverse(int n){
    int rev=0, remainder;
    while (n!=0){
        remainder = n%10;
        rev = rev*10 + remainder;
        n/=10;
    }
    print(rev);
}
```

Program:

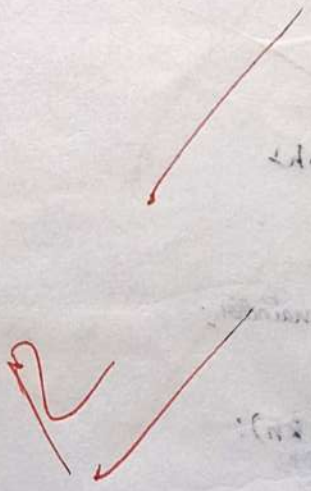
```
#include<stdio.h>
int main(){
    int count=0;
    int rev=0, remainder;
    int n;
    scanf("%d", &n);
    count++;
    count++;
    while (n!=0){
        count++;
        count++;
        remainder = n%10;
        count++
        rev= rev*10 +remainder;
        count++;
        n/=10;
    }
    count++;
```

```
    printf("%d", count);
    return 0;
}
```

Sample Test Case: -

Input	Expected Output	Output
12	11	11
1234	19	19

RESULT: Thus, the program is written and executed successfully using counter Method.



Divide and Conquer

Problem : 1

AIM: To find the number of 0s. Write a program using Divide and Conquer to count the number zeroes in the given array.

Algorithm:

```
function first zero (arr, low, high){
    if (low > high)
        return -1;
    mid = low + (high - low) / 2 ;
    if (arr [mid] == 0 AND (mid == 0 or arr[mid-1] == 1))
        return mid;
    if (arr[mid] == 1){
        return first zero(arr, mid+1, high);
    }
    else
        return first zero(arr, low, mid-1);
}
Main(){
    declare n, i, result;
    declare arr[n];
    Read n;
    for (i=0 ; i<n; i++){
        read arr[i];
    }
    result = first zero (arr, 0, n-1);
    if (result == -1)
        print No zero found;
    else print, no: of zero
        result;
}
```

Program:

```
#include<stdio.h>
int countZeroes (int arr[], int low, int high) {
    if (low>high) {
        return 0;
    }
    int mid = low + (high - low)/2;
    if (arr[mid]==0) {
        return (high - mid +1) + countZeroes (arr, low, mid-1);
    }
    else{
        return countZeroes (arr, mid+1, high);
    }
}
int main() {
    int m;
    scanf ("%d",&m);
    int arr[m];
    for (int i=0; i<m; i++) {
        scanf ("%d", &arr[i]);
    }
    int numZeroes = countZeroes (arr, 0, m-1);
    printf ("%d\n", numZeroes);
    return 0;
}
```

Sample Test Case.

Input	Expected	Opt
5	2	2
1		
1		
1		
0		
0		

RESULT: Thus the program is written and executed successfully to find the number of zeroes in an array.

Problem : 2 Majority Element

Aim: Given an array nums of size n, return the majority element. The majority element is the element that appears more than $\lfloor n/2 \rfloor$ times. You may assume that the majority element always exists in the array.

Algorithm:

```
int majority (int a[], int low, int high) {
    int mid, left, right, count1, count2;
    if (low == high)
        return a[low];
    mid = (low + high) / 2;
    left = majority (a, low, mid);
    right = majority (a, mid+1, high);
    if (left == right) return left;
    count1 = 0;
    count2 = 0;
    for (int i = low; i <= high; i++) {
        if (a[i] == left) count1++;
        if (a[i] == right) count2++;
    }
    if (count1 > count2)
        return left;
    else
        return right;
}
```

program:

```
#include <stdio.h>
int majorityElement (int* nums, int numsSize){
    int majorityCount = numsSize/2;
    for (int i=0; i<numsSize ;i++){
        int count=0;
        for (int j=0 ;j<numsSize ;j++){
            if (nums[j]==nums[i]){
                count++;
            }
        }
        if (count > majorityCount){
            return nums[i];
        }
    }
    return -1;
}
int main(){
    int n;
    scanf("%d",&n);
    int nums[n];
    for (int i=0; i<n; i++){
        scanf("%d", &nums[i]);
    }
    int result = majorityElement (nums, n);
    printf("%d\n", result);
    return 0;
}
```

Sample Test Case:

Input	Output
3	3
3 2 3	

RESULT: Thus, the program is written and executed successfully to find the majority Element of an given array.

Problem 3: Finding Floor Value

Aim: Given an sorted array and a value x, the floor of x is the largest element in array smaller than or equal to. Write divide and conquer algorithm to find floor of x.

Algorithm:

```
int findfloor (int arr[], int low, int high, int x) {
    int floor = -1;
    while (low <= high) {
        int mid = low + (high-low)/2;
        if (arr[mid] == x){
            return arr[mid];
        }
        else if (arr[mid] < x) {
            floor = arr[mid];
            low = mid+1;
        }
        else{
            high = mid-1;
        }
    }
    return floor;
}
```

Program:

```
#include<stdio.h>
int findfloor (int arr[], int low, int high, int x){
    int floor = -1;
    while (low <= high){
        int mid = low + (high - low)/2;
        if (arr[mid] == x){
            return arr[mid];
        }
```

```
        else if (arr[mid] < x) {
            floor = arr[mid];
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }
    return floor;
}
int main() {
    int n, x;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    scanf("%d", &x);
    int floor_value = find_floor(arr, 0, n-1, x);
    printf("%d\n", floor_value);
    return 0;
}
```

Sample Test Case:

Input	Output
6	2
1	
2	
8	
10	
12	
19	
5	

RESULT: Thus, algorithm and program is written and executed successfully to find the floor value of x in an given array.

Problem: "Two elements sum to x"

Aim: Given a sorted array and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If their exist such two elements then return 1, otherwise print as "no".

Algorithm:

```
int findPair(int arr[], int i, int j, int x){
    if (i>=j){
        return 0;
    }
    if (arr[i]+arr[j]==x){
        printf("%d %d", arr[i], arr[j]);
        return 1;
    }
    else if (arr[i]+arr[j]<x){
        return findpair(arr, i+1, j, x);
    }
    else{
        return findpair(arr, i, j-1, x);
    }
}
```

Program:

```
#include<stdio.h>
#include<stdlib.h>
int main(){
    int n, x;
    scanf("%d", &n);
    int *arr[n];
    for
```

```
for (int i=0; i<n; i++) {
    scanf("%d", &arr[i]);
}
scanf("%d", &x);
int i=0, j=n-1;
while (i<j) {
    int sum = arr[i] + arr[j];
    if (sum == x) {
        printf("%d %d\n", arr[i], arr[j]);
        return 0;
    }
    else if (sum < x) {
        i++;
    }
    else {
        j--;
    }
}
printf("No\n");
return 0;
}
```

Sample Test Case

Input	Output
4	4
1	10
4	
8	
10	
14	

RESULT: Thus, algorithm and program was written and executed successfully and outputs the two elements sum to x.

Problem: 5 Implementation Of Quick Sort

Aim: To implement a Quick Sort Algorithm.

Algorithm:

```
quicksort (a, low, high) {
    if (low < high) {
        p = partition (a, low, high);
        quicksort (a, low, p-1);
        quicksort (a, p+1, high);
    }
}
partition (a, low, high) {
    pivot = a[high];
    i = low - 1;
    for (int j = low to high - 1) {
        if (a[j] < pivot) {
            i++;
            swap (a[i], a[j]);
        }
    }
    swap (a[i+1], a[high]);
    return i+1;
}
```

program:

```
#include <stdio.h>
void swap (int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}
int partition (int arr[], int low, int high) {
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot) swap (&arr[i+1], &arr[j]);
    swap (&arr[i+1], &arr[high]);
    return i+1;
}
```

```
void quicksort(int arr[], int low, int high){
    if (low < high){
        int pi=partition(arr, low, high);
        quicksort (arr, low, pi-1);
        quicksort (arr, pi+1, high);
    }
}

int main(){
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i=0; i<n; i++) scanf("%d", &arr[i]);
    quicksort(arr, 0, n-1);
    for (int i=0; i<n; i++) printf("%d ", arr[i]);
    return 0;
}
```

Sample Test Case:

Input	Expected Output	output
5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98

RESULT: Thus the program was written and executed successfully to implement a quick sort algorithm.

GREEDY ALGORITHMS

1 G- Coin Problem

AIM: To write a program to find the minimum number of indian Currency coins | notes required to make a given value v using the available denominations.

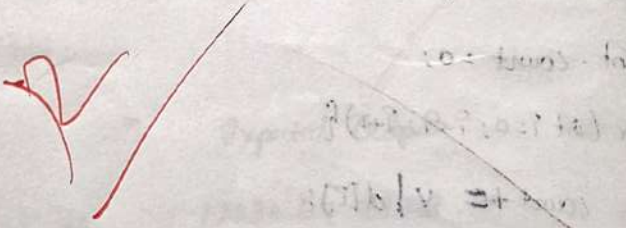
programs

```
#include <stdio.h>
int main() {
    int v;
    scanf("%d", &v);
    int d[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
    int count = 0;
    for (int i=0; i<9; i++) {
        count += v / d[i];
        v %= d[i];
    }
    printf("%d", count);
    return 0;
}
```

Sample Test Case:-

Input	Expected	Got
49	5	5

RESULT: The program was written and executed successfully.



26- Cookies Problem

Aim: Write a program to maximise the number of content childer by assigning cookies of sufficient size based on each child's greed factor.

Program:

```
#include <stdio.h>
int main() {
    int n, b;
    scanf("%d", &n);
    int g[n];
    for (int i=0; i<n; i++) {
        scanf("%d", &g[i]);
    }
    scanf("%d", &b);
    int s[b];
    for (int i=0; i<b; i++) {
        scanf("%d", &s[i]);
    }
    int i=0, j=0, count=0;
    while (i<n && j<b) {
        if (g[i] <= s[j]) {
            count++;
            i++;
            j++;
        }
        else {
            j++;
        }
    }
}
```

```
    printf("%d", count);
    return 0;
}
```

Sample Test Case:-

Input	Expected	Got
2 1 2 3 1 2 3	2	2

RESULT:

The program was written and executed successfully.



3. Burger Problem

Aim:

To write a program to find the maximum distance a person needs to run by sorting the burgers in descending order of calories before eating them.

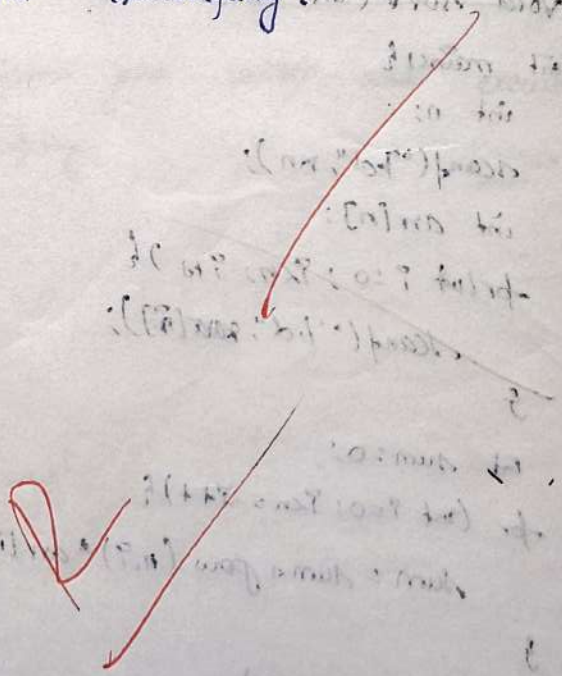
Program:

```
#include <stdio.h>
#include <math.h>
void sort(int arr[], int n){
int main(){
    int n;
    scanf("%d", &n);
    int arr[n];
    for(int i=0; i<n; i++){
        scanf("%d", &arr[i]);
    }
    int sum=0;
    for(int i=0; i<n; i++){
        sum = sum + pow(n,i) * arr[i];
    }
    printf("%d", sum);
}
```

Sample Test Case:-

Test	Input	Expected	Got
Test Case 1	3 1 3 2	18	18
Test Case 2	4 7 4 9 6	389	389
Test Case 3	3 5 10 7	76	76

RESULT: Thus, the program was written and executed successfully.



46 - Array Sum Max Problem.

AIM:

To write the program to maximize the sum of given array

Program:

```
#include <stdio.h>
#include <stdlib.h>
int cmp(const void *a, const void *b){
    return *(int*)a - *(int*)b;
}
int main(){
    int n;
    scanf("%d", &n);
    int a[n];
    for(int i=0; i<n; i++){
        scanf("%d", &a[i]);
    }
    qsort(a, n, sizeof(int), cmp);
    int sum=0;
    for(int i=0; i<n; i++){
        sum += a[i] * i;
    }
    printf("%d", sum);
}
```

Sample Test Case:-

Input	Expected	Got
5	40	40
2		
5		
3		
4		
0		

RESULT:

The program was written and executed successfully.

56. product of array elements - Minimum.

Aim

To write a program to arrange two arrays such that sum of pairwise products is minimized.

Program:

```
#include<stdio.h>
#include<stdlib.h>
int compare1(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}
int compare2(const void *a, const void *b) {
    return (*(int*)b - *(int*)a);
}
int main() {
    int n, sum=0;
    scanf("%d", &n);
    int a[n], b[n];
    for(int i=0; i<n; i++){
      for(int j=0; j<n; j++) {
        qsort(a,n, sizeof(int), compare1)
        qsort(b, n, sizeof(int), compare2)
      }
    }
    for(int i=0; i<n; i++){
        sum += (a[i] * b[i]);
    }
    printf("%d", sum);
    return 0;
}
```

Sample Test Case:-

Input	Expected	Got
3 1 2 8 4 5 6	28	28

RESULT:

Thus the program is executed successfully.

1-DP Playing with Numbers

AIM: To represent the number 'n' using three number 1 and 3 in all possible ways.

Program:

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    long long dp[n+1];
    dp[0]=1
    for(int i=1;i<n;i++){
        dp[i]=dp[i-1]
        if(i>=3)
            dp[i]+=dp[i-3];
    }
    printf("%d",dp[n]);
    return 0;
}
```

Sample Test case:-

Input	Excepted	Got
6	6	6
25	86411	86411
100	24382819596721629	24382819596721629

RESULT: Hence the program was written and executed successfully.

2-DP - Playing with chessboard

AIM: To write an efficient program to reach the bottom-right block on chess board with the given constraints.

Program:

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int a[n][n];
    int dp[n][n];
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            scanf("%d",& a[i][j]);
        }
    }
    dp[0][0] = a[0][0];

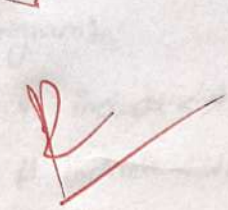
    for(int i=0; i<n; i++){
        for(int j=1; j<n; j++){
            if(dp[i-1][j] > dp[i][j-1])
                dp[i][j] = dp[i-1][j] + a[i][j];
            else
                dp[i][j] = dp[i][j-1] + a[i][j];

    printf("%d", dp[n-1][n-1]);
    return 0;
}
```

Sample Test Case:-

Input	Expected	Got
3	19	
1 2 4		
2 3 4		
8 7 1		
3		
1 3 1	12	
1 5 1		
4 2 1		

RESULT: Thus, the program was written and executed the chessboard play successfully.



8 - DP - Longest Common Subsequence.

AIM: To find the longest common subsequence from two strings.

Program:

```
#include<stdio.h>
#include<string.h>

int main(){
    char s1[100], s2[100];
    scanf("%s", s1);
    scanf("%s", s2);
    int n = strlen(s1);
    int m = strlen(s2);
    int dp[n+1][m+1];
    for(int i=1 ; i<=n; i++){
        for(int j=1 ; j<=m; j++){
            if(s1[i-1]==s2[j-1])
                dp[i][j] = dp[i-1][j-1] +1;
        }
    }
    printf("%d", dp[n][m]);
    return 0;
}
```

Sample Test Case:-

Input	Expected	Got
aab azb	2	2
ABCD ABCD	4	4

RESULT: Thus the program was written and executed successfully to find the longest co subsequence.

4-DP - longest non-decreasing subsequence

Aim: To find the length of longest Non-decreasing subsequence in a given sequence.

Program:

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int a[n], dp[n];
    for (int i=0; i<n; i++){
        scanf("%d", &a[i]);
        dp[i]=1;
    }
    int ans=1;
    for (int i=1; i<=n; i++){
        for (int j=0; j<i; j++){
            if (a[j] <= a[i]){
                if (dp[j]+1 > dp[i]){
                    dp[i] = dp[j]+1;
                }
            }
        }
        if (dp[i] > ans)
            ans = dp[i];
    }
    printf("%d", ans);
    return 0;
}
```

Sample Test Case:-

Input	Expected	Got
9 -1 3 45 2222 3	6	6
7 1 2 2 4 5 7 6	6	6

RESULT:- The program was written and executed successfully.

1- Finding Duplicates - O(n^2) Time complexity, O(1) Space complexity.

Aim: To find the duplicates number from the given array.

Program:

```
#include <stdio.h>
int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i=0; i<n; i++) {
        scanf("%d", &arr[i]);
    }
    for(int i=0; i<n; i++) {
        for (int j=i+1; j<n; j++) {
            if (arr[i] == arr[j])
                printf("%d", arr[j]);
        }
    }
    return 0;
}
```

Sample Test Cases:-

Input	Expected	Got
11 10 9 7 6 5 1 2 3 8 4 7	7	7
5 1 2 3 4 4	4	4
5 1 1 2 3 4	1	1

RESULT: Thus, the program was written successfully and executed.

8. Finding Duplicates O(n) Time Complexity O(1) Space Complexity.

Aim: To find duplicate numbers from the given array.

Program:

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i=0; i<n; i++){
        scanf("%d", &arr[n]);
    }
    int vis arr[100] = {0};
    for(int i=0; i<n; i++){
        if(vis arr[i]==0){
            vis arr[arr[i]]++;
        }
        else
            printf("%d", arr[i]);
    }
    return 0;
}
```

Sample Test Case:

Input	Expected	Got.
11 10 9 7 6 5 1 2 3 8 4 7	7	7
5 1 2 3 4 4	4	4

Result: Thus the program was written and executed successfully.

8- Print Intersection of 2 sorted Arrays

O(m+n) Time Complexity O(1) Space Complexity.

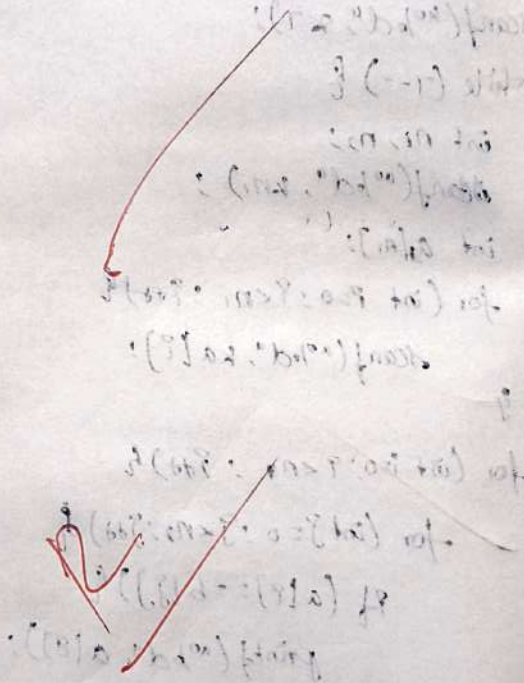
AIM: To find all the elements occur in both the arrays.

Program:

```
#Include<stdio.h>
int main(){
    int T;
    scanf("%d", &T);
    while (T--) {
        int n1, n2;
        scanf("%d", &n1);
        int a[n1];
        for (int i=0; i<n1; i++){
            scanf("%d", &a[i]);
        }
        for (int i=0; i<n1; i++){
            for (int j=0; j<n2; j++){
                if (a[i]==b[j]){
                    printf("%d", a[i]);
                    break;
                }
            }
        }
        printf("\n");
    }
    return 0;
}
```

Test Case:-

Input	Expected	Gpt
1 3 10 17 57	10 57	10 57
6 2 7 10 15 57 246		
1 6 1 2 3 4 5 6	1 6	1 6
2 1 6		



4 - Print Intersection of Two Sorted Arrays.

O(m+n) Time Complexity, O(1) Space Complexity.

Aim: To find all the Elements that occur in both the arrays.

Program:

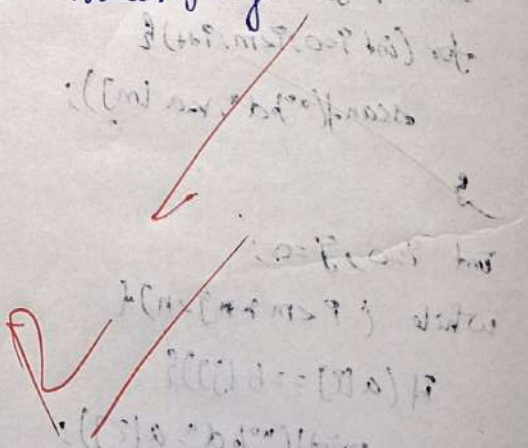
```
#include<stdio.h>
int main(){
  int T;
  scanf("%d", &T);
  while (T--){
    int m,n;
    scanf("%d %d", &m, &n);
    int a[m];
    for (int i=0; i<m; i++){
      scanf("%d", &a[m]);
    }
    int i=0, j=0;
    while (i<m && j<n){
      if (a[i]==b[j]){
        printf("%d", a[i]);
        i++;
        j++;
      }
      else if (a[i]<b[j]){
        i++;
      }
      else{
        j++;
      }
    }
    printf("\n");
}
```

return 0;
}

Test Case:-

Input	Expected	Opt.
1 3 10 17 57	10 57	10 57
6 2 7 10 15 57 246		

RESULT: Thus, the program was written and executed successfully



5. Pair with Difference

$O(n^2)$ Time Complexity, $O(1)$ Space Complexity

Aim: To find wether there exist two indices i, j in array such that $a[i] - a[j] = k$, $i \neq j$

Program:

```
#include<stdio.h>
int main()
{
  int n, k;
  scanf("%d", &n);
  int A[n];
  for (int i=0; i<n; i++)
  {
    scanf("%d", &A[i]);
  }
  scanf("%d", &k);
  for (int i=0; i<n; i++)
  {
    for (int j=i+1; j<n; j++)
    {
      if (A[j] - A[i] == k)
      {
        printf("1");
        return 0;
      }
    }
  }
  printf("0");
  return 0;
}
```

Test Case:-

Input	Expected	Got
8 1 8 5 4	1	1
10 1 4 6 8 12 14 15 20 21 25 1	1	1

RESULT: Thus the program was written and executed successfully.



6 - Pair with Difference - O(n) Time Complexity O(1) Space Complexity.

Aim: To find out wether their exist an indexes i and j in an array such that $a[j] - a[i] = k$, $i != j$.

Program:

```
#include<stdio.h>
int main(){
    int n, k;
    scanf("%d%d", &n, &k);
    int A[n];
    for(int i=0; i<n; i++){
        scanf("%d", &A[i]);
    }
    int i=0, j=1;
    while (i<n && j<n){
        if (i==j){
            j++;
        }
        else if (a[j]-a[i]==k){
            printf("1");
            return 0;
        }
        else if(a[j]-a[i]<k){
            j++;
        }
        else{
            i++;
        }
    }
    printf("0");
    return 0;
}
```

Test Case:-

Input	Expected	Opt
5 1 3 5 4	1	
10 1 2 3 5 11 14 16 24 28 29	0	
0		

RESULT: Thus the program was written and executed successfully.

